

Backend Development – 40 Practice Questions

Each Question Carries 5 Marks

Coding and theory questions only — No MCQs.

1. Create a POST /login route that accepts email and password through req.body. Find the user in a predefined users array, verify the password using bcrypt.compare(), and return appropriate success and error responses.
2. Create a POST /signup route that accepts name, email, and password through req.body. Validate the fields, check whether the email already exists, hash the password using bcrypt.hash(), add the user to a predefined users array, and return an appropriate response.
3. Create a React Login Form with email and password fields. Send the form data to an Express POST /login API using fetch(). Display the success or error message returned by the backend on the webpage.
4. Create a React Signup Form with name, email, and password fields and connect it with an Express POST /signup API. Send the form data using fetch() and display the backend response on the webpage.
5. Create a React frontend and Express.js backend integration for a user registration form. The frontend should send name, email, and password using fetch(), while the backend should receive the data through req.body, validate it, and return a JSON response.
6. Create a JWT-based login API. Receive email and password, find the user in a predefined users array, verify the password using bcrypt.compare(), create a JWT containing the user's id, email, and role, and return the token.
7. Create JWT authentication middleware that reads a token from the request headers, verifies it using jwt.verify(), stores the decoded user information in req.user, and uses next() to continue. Return an appropriate response when the token is missing or invalid.
8. Create a protected Express route /dashboard that can be accessed only when a valid JWT is provided. Use authentication middleware to verify the token and return a dashboard success message.
9. Create a JWT-based login and protected dashboard flow. Implement POST /login to verify credentials and generate a JWT, and GET /dashboard to verify the token and allow access only to authenticated users.
10. Create a React login page connected to a JWT-based Express backend. After successful login, receive the JWT from the backend and use it in the Authorization header while requesting a protected /dashboard route.
11. What is JWT authentication? Explain how JWT is used for authentication and to access protected routes.
12. Create a POST /login route that accepts email and password through req.body. Find the user in a predefined users array, verify the password, return a success message for valid credentials, an error message for invalid credentials, and status 400 when credentials are missing.
13. Create a registration API that accepts name, email, and password through req.body. Check that all fields are provided, check whether the email already exists in a predefined users array, and return an appropriate response.
14. Create an Express.js route /dashboard that implements role-based access control. Assume req.user = { name: 'Rahul', role: 'student' }. Allow access only when the role is 'student'; otherwise return a 403 Forbidden response.
15. Create an Express route /admin that can only be accessed by users whose role is 'admin'. Implement middleware to check the user's role before allowing access.
16. What is middleware in Express.js? Explain how middleware works in the request-response cycle. Write middleware that prints 'Request received' in the terminal for every request.
17. Write an Express middleware that checks whether req.user exists. If the user is not available, return a 401 response; otherwise use next() to continue.
18. Explain the purpose of next() in Express middleware. Write a suitable middleware example that passes control to the next middleware or route.
19. Differentiate between req.body, req.params, and req.query in Express.js. Give one example of each.

- 20.** Consider POST /users?id=10 with the body { 'name': 'Rahul' }. Explain where the values 10 and 'Rahul' are available in Express and write the code to access them.
- 21.** Create an Express API using an array of products. Implement GET /products, POST /products using req.body, and DELETE /products/:id using req.params. Use products with id, name, category, and price.
- 22.** Create a GET /products/:id route that finds a product using the ID received from req.params. Return the product if found; otherwise return an appropriate error response.
- 23.** Create a POST /products route that receives name, category, and price through req.body. Validate the required fields and add the product to a predefined products array.
- 24.** Create a PUT /products/:id route that updates a product in a predefined products array. Use req.params to identify the product and req.body to receive the updated details.
- 25.** Create an Express Student API using an array of students. Implement GET /students, GET /students/:id, POST /students, and DELETE /students/:id.
- 26.** Create a REST API for students using an array. Implement GET /students, POST /students, PUT /students/:id, and DELETE /students/:id.
- 27.** Explain the purpose of the HTTP methods GET, POST, PUT, and DELETE in a REST API. Give one suitable route example for each.
- 28.** Explain the purpose of HTTP status codes 200, 201, 400, 401, 403, and 404. Give one suitable situation where each can be used.
- 29.** Create a React form that accepts name and email and sends the data to an Express POST /users API using fetch(). The backend should receive the data through req.body, display it in the terminal, and return { 'message': 'User data received successfully' }.
- 30.** Create a React frontend and Express.js backend integration for a user form. The frontend should send name and email using fetch(), receive the backend response, and display the response message on the webpage. Use express.json() and cors().
- 31.** Explain why express.json() is required when an Express API receives JSON data through req.body. Give a suitable example.
- 32.** Explain the purpose of CORS in a React frontend and Express backend application. Show how CORS can be enabled in Express.
- 33.** Create a GET /users and POST /users API using a predefined users array. The POST route should receive user data using req.body, validate the data, add the user, and return an appropriate response.
- 34.** Create a complete Express-based user API using an array. Implement GET /users, GET /users/:id, POST /users, PUT /users/:id, and DELETE /users/:id. Use req.params and req.body wherever required.
- 35.** Create a login API using bcrypt. Receive email and password, find the user by email, use bcrypt.compare() to verify the password hash, and return 'Login successful' or 'Invalid email or password'.
- 36.** Create a signup API using bcrypt. Validate name, email, and password, check for an existing email, hash the password using bcrypt.hash(), store the new user in an array, and return a registration success response.
- 37.** Create authentication middleware that verifies a JWT using a secret key. If verification succeeds, store the decoded payload in req.user; if verification fails, return an appropriate 401 response.
- 38.** Create a role-checking middleware that allows only users with role 'admin' to access /admin. Assume authentication middleware has already stored the decoded JWT payload in req.user.
- 39.** Create a complete signup, login, and dashboard backend flow. Signup should hash and store the password, login should verify the password and create a JWT, and dashboard should verify the JWT before allowing access.
- 40.** Create a React frontend for signup and login connected to an Express backend. Implement signup and login APIs, use bcrypt for password verification, generate a JWT after successful login, and use the JWT to access a protected

dashboard route.